

Deep Learning-Based Lunar Crater Detection Using Modern Object Detection Architectures

Scott Nguyen

University of New Mexico ECE 533

Spring 2026

- Evaluated modern deep learning object detection models for automated lunar crater detection
- Compared YOLOv8 Nano, Faster R-CNN, and RetinaNet
- Performed dataset preparation, annotation verification, training, visualization, and evaluation
- YOLOv8 achieved the strongest balance between accuracy, speed, and computational efficiency
- Demonstrated effectiveness of deep learning for planetary surface analysis

Why Lunar Crater Detection Matters

- Planetary surface mapping
- Terrain analysis
- Hazard assessment
- Autonomous aerospace navigation
- Large-scale planetary image analysis

Problem Statement

- Manual crater annotation is time-consuming
- Difficult to scale to large datasets
- Inefficient for automated workflows

Project Contributions

- Developed a complete crater detection pipeline
- Implemented:
 - YOLOv8 Nano
 - Faster R-CNN
 - RetinaNet
- Performed quantitative and qualitative evaluation
- Compared runtime efficiency and localization quality
- Generated prediction visualizations and comparison plots

YOLOv8

- Single-stage detector
- Lightweight and efficient
- Fast inference performance

Faster R-CNN

- Two-stage detector
- Strong localization quality
- Higher computational cost

RetinaNet

- Single-stage detector
- Uses focal loss
- Balanced detection performance

Dataset Description

Dataset Source

<https://www.kaggle.com/datasets/lincolnzh/martianlunar-crater-detection-dataset>

Dataset Contents

- Lunar and Martian imagery
- Crater bounding box annotations
- YOLO-formatted labels

Dataset Splits

- Training dataset
- Validation dataset
- Testing dataset

Dataset Distribution

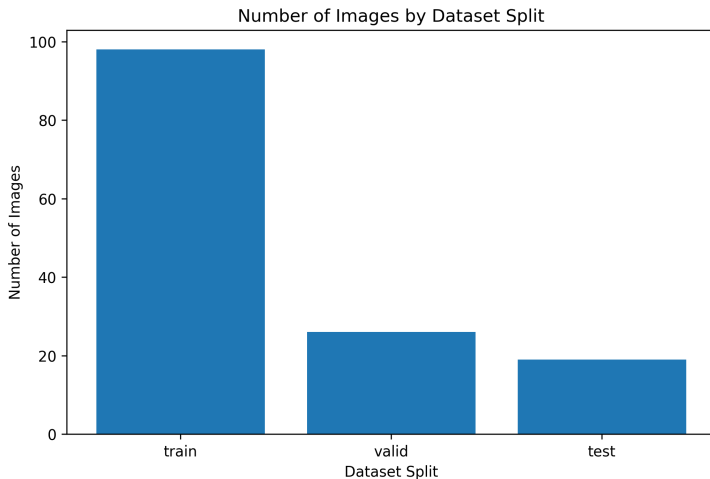


Figure: Distribution of images across dataset splits.

Annotation Distribution

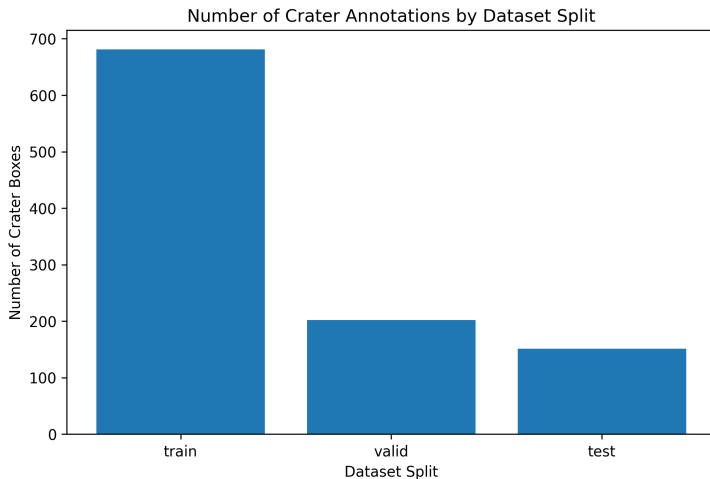
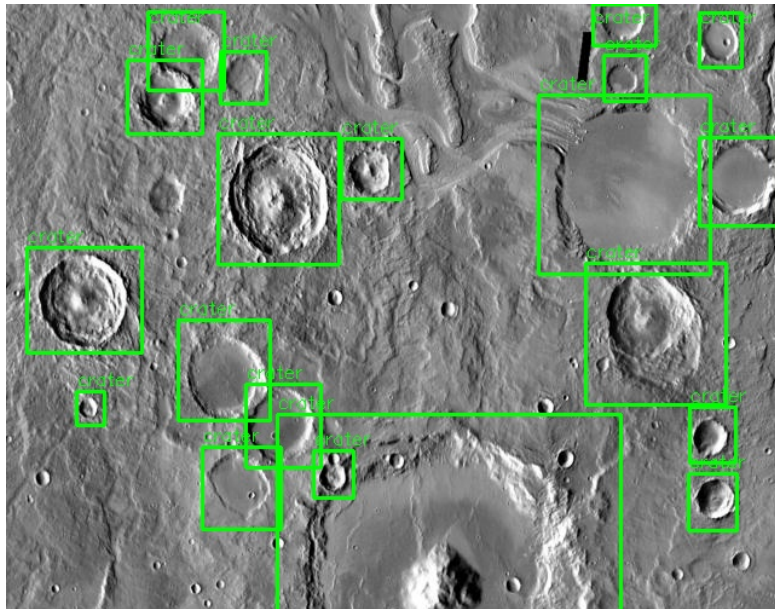


Figure: Distribution of crater annotations across dataset splits.

Ground Truth Visualization



Experimental Workflow

- 1 Dataset initialization
- 2 Dataset inspection
- 3 Annotation verification
- 4 Ground truth visualization
- 5 Model training
- 6 Prediction generation
- 7 Quantitative evaluation
- 8 Timing analysis
- 9 Comparative analysis

High-Level Pipeline

- Load crater dataset
- Verify annotations
- Generate ground truth visualizations
- Train YOLOv8, Faster R-CNN, and RetinaNet
- Save weights and training logs
- Generate crater predictions
- Measure runtime performance
- Compare localization and efficiency

Model Configuration

YOLOv8 Nano

- Single-stage detector
- Fast crater detection
- Efficient runtime

Faster R-CNN

- Two-stage detector
- Strong localization quality

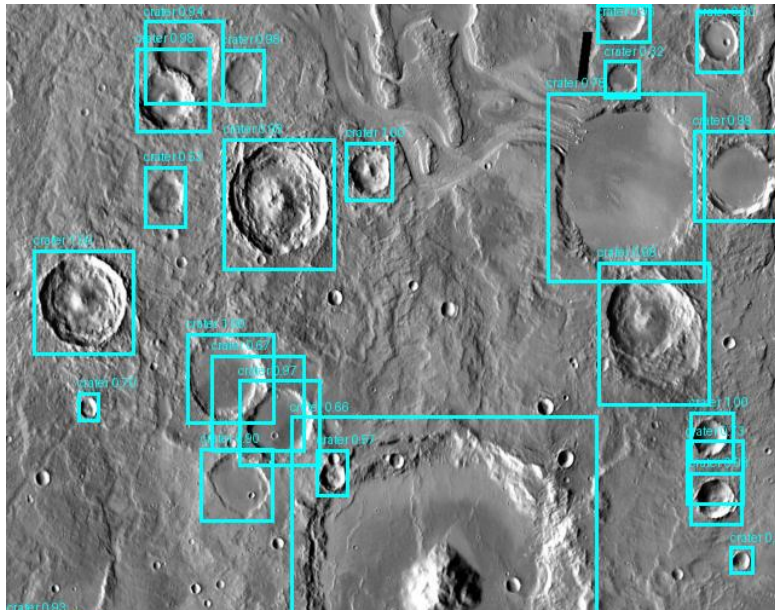
RetinaNet

- Single-stage detector
- Balanced performance

Training Configuration

- 50 training epochs
- 640×640 image resolution
- Batch size of 8
- Tesla T4 GPU acceleration
- Consistent dataset splits across all models

RetinaNet Prediction



Overall Model Comparison

- **YOLOv8:** high speed, high efficiency, strong localization
- **Faster R-CNN:** low speed, low efficiency, very strong localization
- **RetinaNet:** medium speed, medium efficiency, strong localization

Main Observation

YOLOv8 achieved the strongest balance between:

- crater localization quality
- inference speed
- computational efficiency

Accuracy Comparison

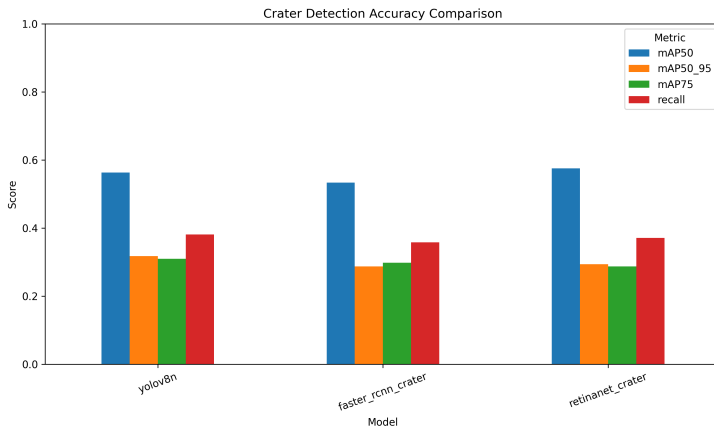


Figure: Accuracy metric comparison across evaluated models.

Training Time Comparison

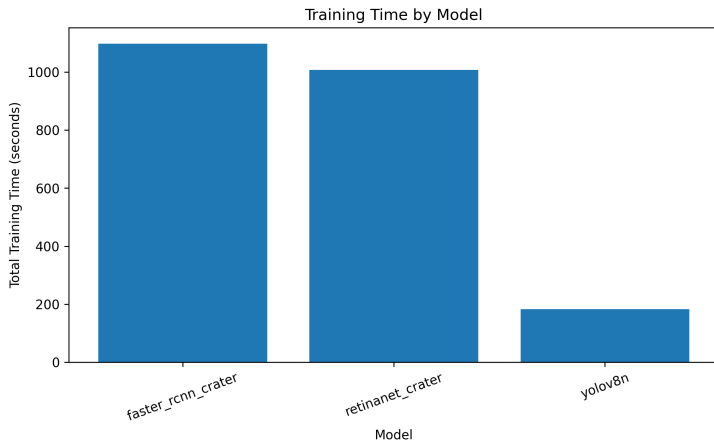


Figure: Training time comparison across evaluated models.

Inference Time Comparison

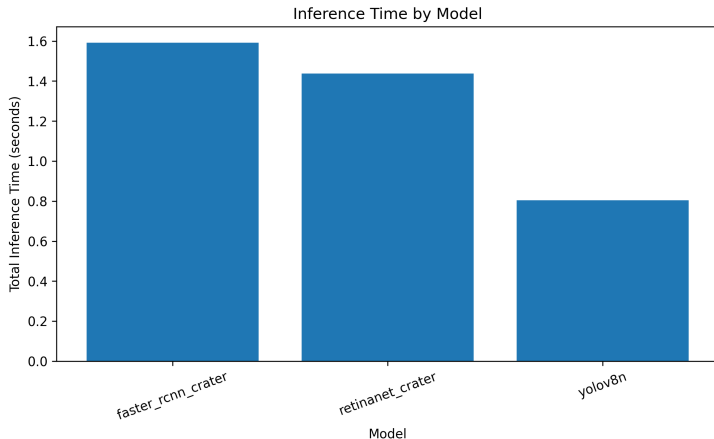


Figure: Inference runtime comparison across evaluated models.

Visual Performance Summary

- **YOLOv8:** best runtime-performance balance
- **Faster R-CNN:** strong localization quality but higher computational cost
- **RetinaNet:** balanced overall performance

What Worked

- YOLOv8 achieved strong localization with fast runtime
- Faster R-CNN produced accurate crater localization
- RetinaNet handled crater-background imbalance effectively

Limitations

- Limited dataset size
- Overlapping crater boundaries
- Terrain and illumination variation
- Small crater localization difficulty

Summary

- Developed a complete crater detection pipeline
- Evaluated YOLOv8, Faster R-CNN, and RetinaNet
- Compared detection accuracy and computational efficiency
- YOLOv8 achieved the strongest overall balance

Future Work

- Larger crater datasets
- Higher-resolution imagery
- Segmentation-based crater detection
- Embedded aerospace deployment optimization

Google Colab Implementation

<https://drive.google.com/drive/folders/180nwDg1re-HRRi3NxY0z8zSU-ZSPvP3b>

Includes

- Dataset preparation scripts
- YOLOv8 training pipeline
- Faster R-CNN training pipeline
- RetinaNet training pipeline
- Evaluation scripts
- Prediction visualizations
- Accuracy and timing plots

References

- ① Ultralytics YOLOv8 <https://github.com/ultralytics/ultralytics> ""
- ② TorchVision Object Detection Models
<https://pytorch.org/vision/stable/models.html>
- ③ Lunar and Martian Crater Dataset
<https://www.kaggle.com/datasets/lincolnzh/martianlunar-crater-detection-dataset>
- ④ PyTorch Documentation
<https://pytorch.org/>
- ⑤ KaggleHub Documentation
<https://github.com/Kaggle/kagglehub> ""

Thank You

Questions?